WALNUTS:

Per-leapfrog step size adaptation for HMC

Bob Carpenter

Center for Computational Mathematics

Flatiron Institute

Bayesian statistics with Monte Carlo

- Bayesian **posterior inference** given data y with parameters θ:

$$\begin{aligned} \widehat{\theta} &= \mathbb{E}[\Theta \mid y] && \text{[parameter estimates]} \\ \Pr[E \mid y] &= \mathbb{E}[\mathrm{I}_E(\Theta) \mid y] && \text{[event probability estimates]} \\ p(\widetilde{y} \mid y) &= \mathbb{E}[p(\widetilde{y} \mid \Theta) \mid y] && \text{[posterior predictive density]} \end{aligned}$$

- Expectations involve **high-dimensional integrals**.

$$\mathbb{E}[f(\Theta) \mid y] = \int_\Theta f(\theta) p(\theta \mid y) \mathrm{d}\theta.$$

- (Markov chain) **Monte Carlo solves integrals** with $\theta^{(m)} \sim p(\theta \mid y)$.

$$\mathbb{E}[f(\Theta) \mid y] \approx \frac{1}{M} \sum_{m=1}^{M} f(\theta^{(m)})$$

Why NUTS?

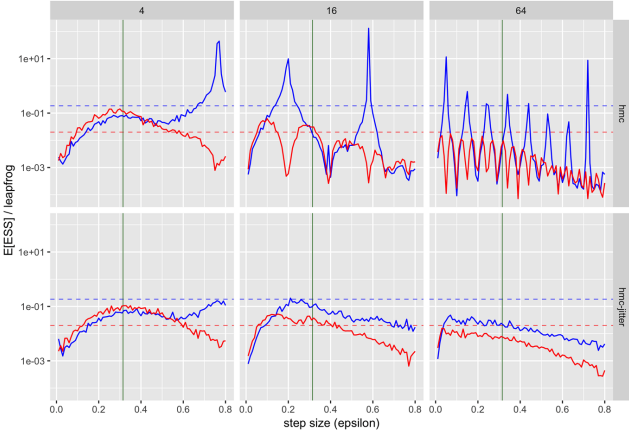
- Hamiltonian Monte Carlo (HMC, 1985), a Markov chain Monte Carlo (MCMC) method that **scales well in dimension**.
  - **integrated autocorrelation time***: $\mathcal{O}\left(D^{1/4}\right)$ **memory**: $\mathcal{O}(D)$

    *: times "constant" based on condition number (Langmore et al. 2020)
  - Hamiltonian **gradient flow** overcomes diffusive random walk behavior.
- Why wasn't HMC in wide use until the **no-U-turn sampler** (NUTS, 2012)?
  - NUTS in Stan, PyMC, NumPyro, Turing.jl, Blackjax, NIMBLE, ADMB, etc.
- **Double whammy**:
  - HMC is hard to tune; NUTS brought **self tuning** and **general software**.
  - gradients are hard for applications; **automatic differentiation** makes it easy.

What is HMC and why is it hard to tune?

- Fictitious physical system with **position** $\theta \in \mathbb{R}^D$ and **momentum** $\rho \in \mathbb{R}^D$:
  - **Potential** energy: $U(\theta) = -\log p(\theta)$ for **target density** $p(\theta)$
  - **Kinetic** energy: $K(\rho) - \log \mathrm{normal}(\rho \mid 0, M)$
  - **Hamiltonian** (total energy): $H(\theta, \rho) = U(\theta) + K(\rho), \quad p(\theta, \rho) = \exp\left(-H(\theta, \rho)\right)$
- **HMC iteration** from position $\theta(0)$:
  - **Sample momentum**: $\rho(0) \sim \mathrm{normal}(0, M)$
  - **Evolve Hamiltonian exactly**: $(\theta(0), \rho(0))$ to $(\theta(t), \rho(t))$, return $\theta(t)$.
- In practice, can't solve exactly, so discretize.
  - **Simulate:** Take L leapfrog steps of size $\epsilon = t/L$ to $(\theta'(t), \rho'(t)) \approx (\theta(t), \rho(t))$.
  - **Metropolize:** Return $\theta'(t)$ with probability $1 \wedge \frac{p(\theta'(t),\ \rho(t)')}{p(\theta(0),\ \rho(0))}$; else return $\theta(0)$.

Harmonics in HMC



- **Target**: 1000-dim standard normal
- **y-axis** (log): ESS; **x-axis**: step size
- **blue**: $\mathbb{E}[\Theta]$; **red**: $\mathbb{E}[\Theta^2]$
- **top**: HMC; **bottom**: jittered time HMC
- **solid**: HMC, **dashed**: NUTS

Leapfrog integrator simulates trajectories

- **Leapfrog step** from $(\theta(t), \rho(t))$ with step ϵ, mass M

$$\begin{aligned}
\rho(t+1/2) &= \rho(t) + \epsilon/2 \cdot \nabla \log p(\theta(t)) \\
\theta(t+1) &= \theta(t) + \epsilon \cdot M^{-1} \cdot \rho(t+1/2) \\
\rho(t+1) &= \rho(t+1/2) + \epsilon/2 \cdot \nabla \log p(\theta(t+1))
\end{aligned}$$

- **Stable** if $\epsilon < 2/\sqrt{\lambda^{\max}}$, where $\lambda^{\max}$ is max eigenvalue of Hessian,

$$H(\theta) = \nabla\nabla^\top - \log p(\theta)$$

  - **local error**: $\mathcal{O}(\epsilon^3)$
  - **global error**: $\mathcal{O}(\epsilon^2)$

Multinomial HMC

- **Each iteration** from previous position $\theta(0)$:
  - generate new momentum $\rho(0) \sim \mathrm{normal}(0, M)$
  - generate **random steps forward** $F \sim \mathrm{uniform}(\{0, \ldots, L\})$
  - take $B = L - F$ leapfrog **steps backward** and F forward from $(\theta(0), \rho(0))$
  - yields $(\theta(n), \rho(n))$ for $-B \leq n \leq F$ (including initial)
  - **randomly select next state** $(\theta(n), \rho(n))$

    with probability **proportional to density**, $\propto p(\theta(n), \rho(n))$
- **No rejection**, **no harmonics**, and **no bad luck** due to error $\pm H(\theta(0), \rho(0))$
- Straightforward to prove **detailed balance** (cf. our second *GIST* paper)

Naive no-U-turn sampler (NUTS)

- Go forward or backward uniformly at random,
- doubling number of steps each time,
- until there is a U-turn between ends (i.e., ends heading toward each other).
- But, if there is a sub-U-turn in the new states, throw away new doubling.
  - inefficient, but necessary for detailed balance
- select next state proportional to density (a la multinomial HMC)

- Original naive NUTS: slice sample rather than multinomial sample

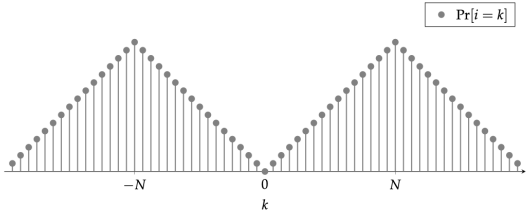
Biased-progressive NUTS

- Keep a selected state, initially $z(0) = (\theta(0), \rho(0))$
- At each doubling from M to $2M$ states:
  - Extend $z(1), \ldots, z(M)$ to $z(1), \ldots, z(M), z(M+1), \ldots, z(2M)$
  - Metropolis **probability of updating state**:

$$1 \wedge \frac{p(z(M+1)) + \cdots + p(z(2M))}{p(z(1)) + \cdots + p(z(M))}$$

  - If Hamiltonian simulation perfect, probability is 1.
  - If updating, **select new state** $z(m)$ with probability **proportional to density**, $\propto p(z(m))$, from among candidate states $z(M+1), \ldots, z(2M)$.
- Return final selected item.

Biased-progressive vs. naive jump distance



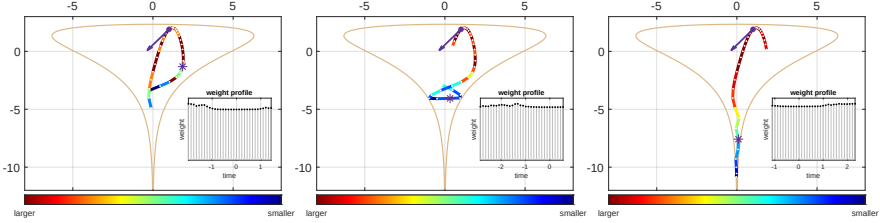
- Assuming **perfect simulation** of the Hamiltonian trajectory, each state equally likely.
- **Naive** NUTS and multinomial sample **uniformly** along trajectory.
- **Biased-progressive** NUTS samples **as above** ($N = 2^M$ total steps)

Preconditioning with a mass matrix

- The positive-definite **mass matrix** M acts as a **global preconditioner**
- If $\Theta \sim \mathrm{normal}(\mu, \Sigma)$, then $M = \Sigma^{-1}$ is a **perfect preconditioner**
  - i.e., mass-adjustment samples like standard normal
- NUTS uses **fixed warmup iterations** with unit mass matrix before sampling to estimate $\mathrm{cov}[\Theta]$
- **Chicken-and-egg** problem:
  - need to sample efficiently to estimate mass matrix effectively
  - need good mass matrix to sample efficiently
- There are **better mass matrix estimators** than we used for Stan (see below).

The curse of varying geometry

- If the Hessian varies, **global preconditioning fails**.
  - e.g., **Neal's funnel**: condition varies 0 to 1000+, eigenstructure varies
- By **adjusting step size** lower in neck (color), **WALNUTS succeeds**



Gibbs self tuning (GIST)

- Couple **auxiliary tuning** variables to position/momentum
  - e.g., $\alpha = L, \epsilon, M$ (number of steps, step size, mass matrix) for HMC
- **Gibbs sample** tuning parameters given state $\alpha \sim p(\alpha \mid \theta, rho)$
  - free to choose reasonable conditional distribution
- Use tuning parameters α with NUTS to **propose** $\theta(t), \rho(t)$
- Accept probability: $\Pr[\text{accept } \theta(t), \rho(t)] = \underbrace{\frac{p(\theta(t), \rho(t))}{p(\theta(0), \rho(0))}}_{\text{Metropolis}} \cdot \underbrace{\frac{p(\alpha \mid \theta(t), \rho(t))}{p(\alpha \mid \theta(0), \rho(0))}}_{\text{correction}}$
- Bou-Rabee, N., Carpenter, B., and M. Marsden. 2024. **GIST: Gibbs self-tuning for locally adaptive Hamiltonian Monte Carlo**. *arXiv*:2404.15253.

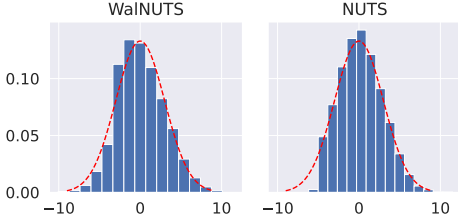
GIST II

- We applied GIST to step size tuning (Bou-Rabee, Marsden, Kleppe, me)
- Start at step size $\epsilon = \epsilon^{\max}$
- While selecting steps L w. NUTS doesn't preserve Hamiltonian within τ between any two states on trajectory
  - $\epsilon = \epsilon/2$
- **Correction is 1** if same number of halvings forward and backward; else reject.
  - GIST technically allows further randomization around halvings.

- Bou-Rabee, N., Carpenter, B., Kleppe, T.S. and Marsden, M., 2025. **Incorporating local step-size adaptivity into the No-U-Turn Sampler using Gibbs self tuning**. *Journal of Chemical Physics*.

GIST III: WALNUTS

- Just like GIST II, but **adjust every leapfrog step**
  - accept correction is product of local corrections



- **Et voila!**—this is log scale in Neal's funnel (marginal $\mathrm{normal}(0, 3)$)

GIST for mass-matrix estimation

- **Local mass matrix** adaptation.
- For **log concave** can use GIST with inverse Wishart distribution around negative Hessian of log density
  - degrees of freedom determined by variability of Hessians
- **Low rank plus diagonal** reduces $\mathcal{O}(N^3)$ to $\mathcal{O}(K^2N)$.
- Black-box Riemannian HMC tunes locally based on (positive-definite conditioned) Hessian
  - Implicit integrators $\mathcal{O}(N^3)$ and hard to tune
  - Explicit interators only require $\mathcal{O}(N^2)$ Hessian-vector products, but just as hard to tune

Nutpie-style mass matrix estimation

- WALNUTS needs: **static mass matrix** and **static macro step size**
- **Nutpie** (from Adrian Seyboldt of PyMC Labs) is state of the art
- Mass matrix M chosen to minimize **Fisher divergence** from preconditioned density to a standard normal

$$\widehat{M} = \arg\min_M \mathrm{D}_F[q_M \,||\, \mathrm{normal}(0,1)],$$

  where q_M is target density preconditioned with M

- Fisher divergence measures **gradient discrepancy**,

$$\mathrm{D}_F[q \,||\, p] = \int_{\mathbb{R}^D} q(\theta) \cdot ||\nabla \log q(\theta) - \nabla \log p(\theta)||_2^2 \, \mathrm{d}\theta.$$

Minimizing Fisher divergence given draws

- Same approach across **matrix forms**: diagonal, low-rank plus diagonal, dense
- Geometric mean of inverse covariance of draws and covariance of scores
  - **score function**: $S(\theta) = \nabla \log p(\theta)$
  - **motivation**: $\Theta \sim \text{normal}(\mu, \Sigma)$ implies $\text{cov}(\Theta) = \text{cov}^{-1}(S(\Theta)) = \Sigma$
  - **Geometric mean** is midpoint of geodesic in **affine-invariant Riemannian manifold** (AIRM) of symmetric positive-definite matrixes
- **Much faster to converge** than using only draws (e.g., NUTS)

Details of Fisher information minimization

- Suppose a **block of approximate draws** gathered during warmup

$$\theta^{(1:N)} = \theta^{(1)}, \ldots, \theta^{(N)} \sim p(\theta)$$

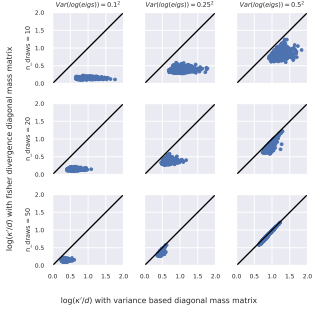
- Nutipe **estimates mass matrix** as

$$\widehat{M} = \mathrm{cov}^{-1}\left(\theta^{(1:N)}\right) \# \mathrm{cov}\left(S(\theta^{(1:N)})\right),$$

- **AIRM geometric mean**: $A \# B = A^{1/2} \left(A^{-1/2}\, B\, A^{-1/2}\right)^{1/2} A^{1/2}$
  - **diagonal case**: $A \# B = (A \cdot B)^{1/2}$ applied elementwise
- Nutpie takes overlapping blocks of $N = 100$ draws

Fisher vs. variance-based mass matrices

- Fisher outperforms covariance by median factor 1.3 over 100+ `posteriordb` models; 4.0 for low-rank plus diagonal
- Log condition/dim; 1000 random spectra w. 10, 20, and 50 i.i.d. draws:



Online Nutpie

- Nutpie **cannot converge** with a fixed window of size $N = 100$
- Stan converges due to **exponentially increasing** blocksizes, e.g., $N = 20, 40, 80, \ldots$
- **Online Nutpie** discounts past to mimic Stan: $\lambda_n = 1/(\text{offset} + n)$
  - default $\text{offset} = 4$ reduces early variance
- Compute **weighted means** and **weighted variances**:
  - $\text{count}(n) = \lambda_n \cdot \text{count}(n-1) + 1$
  - $\text{mean}(n) = \left(\lambda_n \cdot \text{mean}(n-1) + \theta^{(n)}\right) / \text{count}(n)$
  - $\text{var}(n)$ same idea, weighted sum of square diffs from mean
- Implement in **constant time** per iteration w. **weighted Welford algorithm**

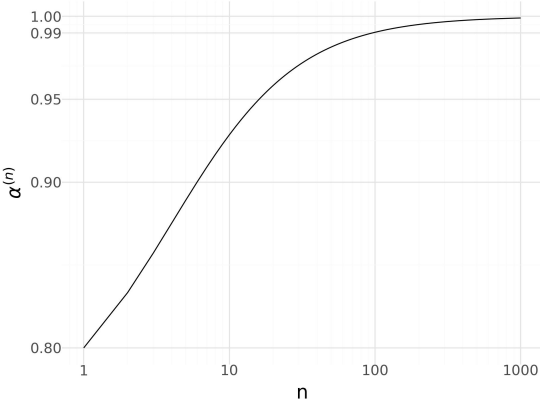
Initial inverse mass matrix

- Stan initializes with **unit** inverse mass matrix.
- Nutpie initializes with an approximation
- For $\Theta \sim \mathrm{normal}(\mu, \Sigma)$, $\mathbb{E}[S(\Theta) \cdot S(\Theta)^\top] = \Sigma$ (Fisher information)
- Use **one-sample MCMC** approximation (or more)
- If $\theta^{(0)}$ is initialization,

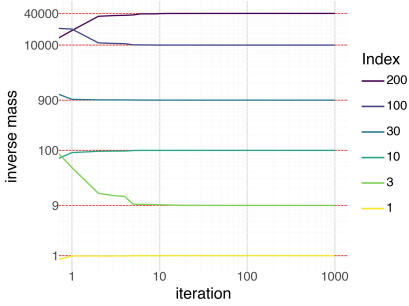
$$M(0) = S(\theta^{(0)})\, S^\top(\theta^{(0)})$$

- Much **more effective** than unit in practice.

Discount schedule; $\mathbf{offset} = 4$



Performance: $\mathbf{normal}(\theta \mid 0, \mathbf{diagMat}([1, 2, 4, \ldots, 200]))$



- **Converges** in **10 iterations**!

Step-size tuning via SGD

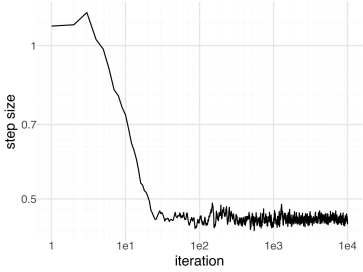
- Tuning parameter: **target acceptance rate** $\rho \in (0, 1)$
- Acceptance rate **monotonic** in step size (lower step, higher accept)
- **Observed acceptance rate** $\alpha_n \in (0, 1)$ at MCMC step n
- **Quadratic loss**: $\mathrm{loss}_\rho(\alpha_n) = \frac{1}{2}(\alpha_n - \rho)^2$
- Given **observed acceptance** α_n stochastic gradient is

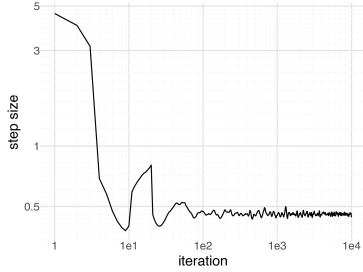
$$\nabla \mathrm{loss}_\rho(\alpha_n) = \alpha_n - \rho$$

SGD algorithms: Adam vs. Dual averaging

- SGD algorithm is **modular**, thus easy to swap
- NUTS used **dual averaging**; 2010 state of the art
  - dual averaging uses stochastic mean to reduce variance
- WALNUTS uses **Adam**; 2023 state of the art SGD
  - Adam estimates a regularized "momentum" for acceleration
- For (WAL)NUTS, Adam is **faster to converge** and **lower variance**
  - even w. Matt Hoffman's **regularization hack** of dual averaging for NUTS
- **Muon** may be the 2026 state-of-the-art SGD

Adam faster, more stable





- **log scale** x-axis and y-axis; **different ranges** on y-axis (1 vs. 5)
- Faster step adapt accelerates **burn-in** and **mass matrix adaptation**.

Lock-free, parallel convergence monitoring

- $\widehat{R}$ **statistic** converges to 1 for chains with same stationary distribution
- **One chain per thread** accumulating mean and variance using Welford
- **SPSC parallelism**: single producer (chain), single consumer (monitor)
- **Monitor thread** reads mean/variance/count from chains
- Monitor computes $\widehat{R}$; terminates chains if below threshold threshold.
- SPSC implemented with **lock-free atomics** (`std::atomic`) in C++11x
  - mean/variance/count packed single precision in 12 bytes
- Monitor **only log density** $\log p(\theta)$—usually slower than single parameters

Lock-free, parallel adaptation monitoring

- We're adapting a **step size** (scalar) and a **diagonal mass matrix** (vector).
- Use a **triple buffer** for lock-free SPSC parallelism queue.
- **Step size convergence** if each chain's step size is within tolerance of the geometric mean across chains.
- **Mass matrix convergence** if each chain's standardized (vs. geometric mean) L2 norm is within tolerance of the geometric mean of chains
- **No measurable overhead** on my Mac Studio
  - M3-ultra CPU; 20 performance cores; 256 GB **integrated RAM**
  - **Mac ARM 5+ times faster** than Intel for **memory-intensive parallelism**

Collaborators and references

- **WALNUTS GitHub**: `flatironinstitute/walnuts`
- **Nutpie GitHub**: `pymc-devs/nutpie`
- Nawaf Bou-Rabee, Bob Carpenter, Tore Kleppe, Sifan Liu. 2026. **WALNUTS: The within-orbit adaptive leapfrog no-U-turn sampler**. *JMLR* (*arXiv* 2506.18746).
- Adrian Seyboldt, Eliot Carlsen, Bob Carpenter. 2026. **Preconditioning Hamiltonian Monte Carlo by minimizing Fisher Divergence**. In progress.

Bou-Rabee

Kleppe

Liu

Seyboldt

Carlsen